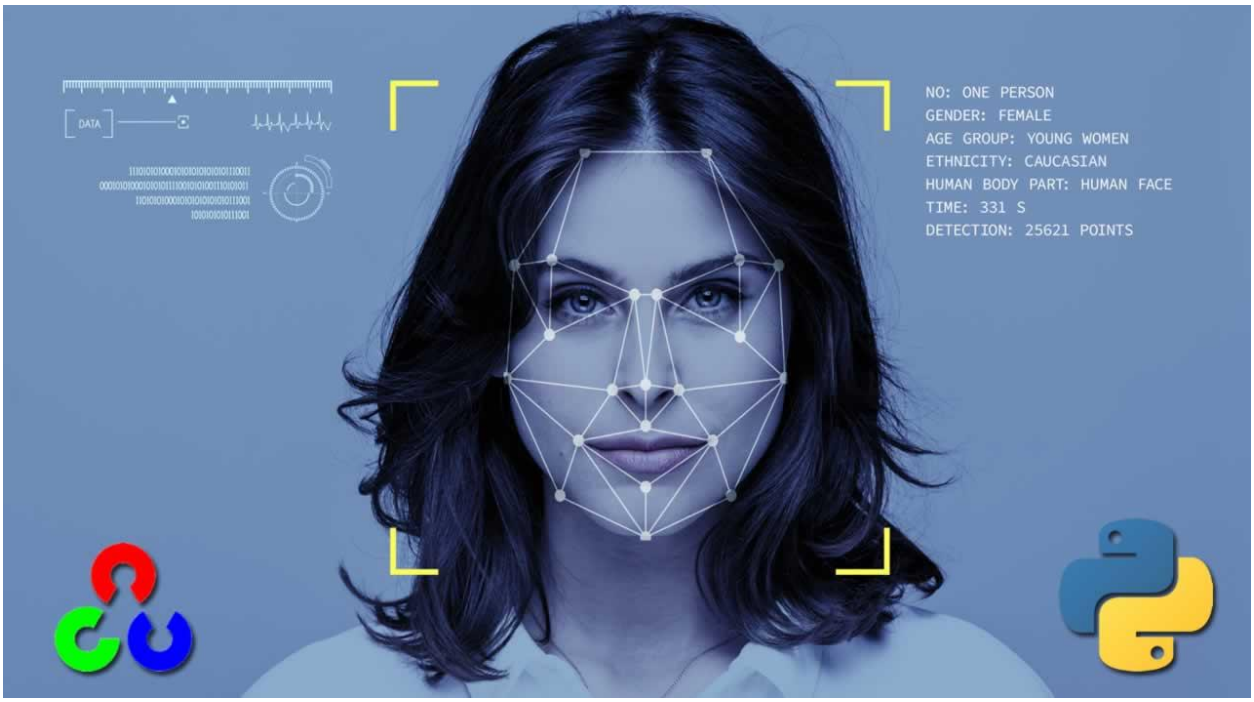
Alexandria University
Faculty of Engineering
Computer and Communication Program

CC484 Pattern Recognition

Assignment One: Face Recognition

Project Link: Colab Notebook



Name	ID
Omar Walied Mohamed	7058
Hend Khaled AbdelHamid M. Aly	6986
Habiba Mohamed Hefny	6939

Table of Contents

- **What is Face Recognition Problem?**

Face recognition means that for a given image, we can tell the subject id. In the given dataset, we have 40 subjects, each of 10 images, each image of size 92*112.

- **Problem Description:**

First. downloaded the dataset from kaggle website.

Second. converted every image into a vector of 10304 values (image size).

Third. Generate the date matrix D and the label vector Y.

Fourth. Apply PCA Classification.

Fifth. Apply LDA Classification.

Sixth. Set the number of neighbors in the K-NN classifier to 1,3,5,7. To apply classifier tunning.

Seventh. Get another dataset containing non-face images to compare it with the face images dataset.

Eighth. Changed the number of instances per subject to 7 and keep 3 instances per subject for testing, then compared the results with the results of 50% split.

Ninth. Used Incremental PCA (IPCA) as a variation of PCA other than the one used in the fourth step.

Tenth. Used Regularized LDA (RDA) as a variation of LDA other than the one used in the fifth step.

Step 1: upload dataset to colaboratory

```
[5] from google.colab import drive
    drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Step 2,3:

```
[6] from PIL import Image
    import numpy as np
    import os
```

using the dataset in generating the data matrix and label vector then split the dataset into training abd test sets (1,2,3)

```
parent_folder = '/content/drive/MyDrive/PR/assignment1'

width = 92
height = 112
#data array
D = np.zeros((400, 10304))
#centered trainig data array
DTraincentered = np.zeros((200, 10304))
#centered test data array
DTestcentered = np.zeros((200, 10304))
#keep track of the number of test and train images processed
testc=0
trainc=0
#training image data array
Dtrain = np.zeros((200, 10304))
#testing image data array
Dtest = np.zeros((200, 10304))
#labels array
Y = np.zeros(400)
#training image labels array
Ytrain = np.zeros((200, 1))
#testing image labels array
Ytest = np.zeros((200, 1))

#loop iterates over the parent folder, where we hae 40 subdirectories each representing a different subject
for sub in range(1,41):
  #sets the path to the subdirectory for the current subject
  folder_path = f"{parent_folder}/s{sub}"
```

```
    #loop iterates over the images in each subject's subdirectory, we have 10 images per subject
    for i in range(1,11):
      #sets the path of current image
      filename = f"{folder_path}/{i}.pgm"
      image = Image.open(filename)
      #convert image to numpy array
      image_array = np.array(image)
      #flatten the image and store it in data array
      D[(sub-1)*10+(i-1)] = image_array.flatten()
      #store the label of the image in labels' array
      Y[(sub-1)*10+(i-1)] = sub
      #if odd -> assign the data as test data
      #if even -> assign the data as train data
      if (i % 2) != 0:
        Dtest[testc] = D[(sub-1)*10+(i-1)]
        Ytest[int(testc)] = Y[(sub-1)*10+(i-1)]
        testc=testc+1
      else:
        Dtrain[trainc] = D[(sub-1)*10+(i-1)]
        Ytrain[int(trainc)] = Y[(sub-1)*10+(i-1)]
        trainc=trainc+1

#subtract the mean of the column from every element in that column
#used for train data
for i in range(0,10304):
  DTraincentered[:,i]=Dtrain[:,i]-Dtrain[:,i].mean()
#similar as the previous loop, but this loop is for test data
for i in range(0,10304):
  DTestcentered[:,i]=Dtest[:,i]-Dtrain[:,i].mean()
```

Step 4,6:

```
#4
# Compute the covariance matrix of the centered training data
covarianceMatrix = np.cov(DTraincentered.T)
# Compute the eigenvalues and eigenvectors of the covariance matrix
eigenvalues, eigenvectors = np.linalg.eigh(covarianceMatrix)
# sort the absolute values of the eigenvalues in descending order
#and return the indices that would sort the eigenvalues in this order.
idx = np.argsort(-np.absolute(eigenvalues))
eigenValues = eigenvalues[idx]
eigenVectors = eigenvectors[:,idx]
```

```
R=np.zeros(4)
#compute the number of principal components required to explain certain percentages of the total variance in a dataset.
#every loop loops through the eigenvalues and calculate the percentage of total variance
#then check if this percentage is greater than the target percentage
#then set the corresponding R to the computed percentage of total variance

for i in range(0,10304):
  explained=eigenValues[0:i].sum()/eigenValues.sum()
  if explained > 0.8:
    R[0]=i
    break
for i in range(i,10304):
  explained=eigenValues[0:i].sum()/eigenValues.sum()
  if explained > 0.85:
    R[1]=i
    break
for i in range(i,10304):
  explained=eigenValues[0:i].sum()/eigenValues.sum()
  if explained > 0.9:
    R[2]=i
    break
for i in range(i,10304):
  explained=eigenValues[0:i].sum()/eigenValues.sum()
  if explained > 0.95:
    R[3]=i
    break
```

```
#U1,U2,U3,U4 are 2D arrays each containing a subset of the columns of eigenvectors
#the arrays contain the eigenvectors of the covariance matrix of the training data
# sorted in descending order of their corresponding eigenvalues
#those arrays are used to construct the projection matrices
U1=eigenVectors[:,:int(R[0])]
U2=eigenVectors[:,:int(R[1])]
U3=eigenVectors[:,:int(R[2])]
U4=eigenVectors[:,:int(R[3])]
```

```
#projection of centered training data onto that eigenvector
#taking the dot product of the centered training data with the corresponding eigenvector
#the transpose of the projection matrix is used to transform the training data into the new coordinate system
projection_matrix = U1
projected_data1_real = np.dot(projection_matrix.T, DTraincentered.T)
projection_matrix = U2
projected_data2_real = np.dot(projection_matrix.T, DTraincentered.T)
projection_matrix = U3
projected_data3_real = np.dot(projection_matrix.T, DTraincentered.T)
projection_matrix = U4
projected_data4_real = np.dot(projection_matrix.T, DTraincentered.T)
```

```
#same as the previous cell, but this is for test data.
projection_matrix = U1
projected_data5_real = np.dot(projection_matrix.T, DTestcentered.T)
projection_matrix = U2
projected_data6_real = np.dot(projection_matrix.T, DTestcentered.T)
projection_matrix = U3
projected_data7_real = np.dot(projection_matrix.T, DTestcentered.T)
projection_matrix = U4
projected_data8_real = np.dot(projection_matrix.T, DTestcentered.T)
```

```
#create an instance of the classifier with k=1 used as the classification model
#fit the training data to the model along with the corresponding class labels Ytrain
#calculate the accuracy of the trained model on the test data and the corresponding class labels Ytrain
#the accuracy is calculated as the number of correct predictions divided by the total no of predictions

#here the number of neighbors is 1
#the process is repeated four times for four different datasets using same no of neighbors

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data1.T, Ytrain)
print("Accuracy with R = ", R[0], ": ", knn.score(projected_data5.T, Ytest))

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data2.T, Ytrain)
print("Accuracy with R = ", R[1], ": ", knn.score(projected_data6.T, Ytest))

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data3.T, Ytrain)
print("Accuracy with R = ", R[2], ": ", knn.score(projected_data7.T, Ytest))

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data4.T, Ytrain)
print("Accuracy with R = ", R[3], ": ", knn.score(projected_data8.T, Ytest))
```

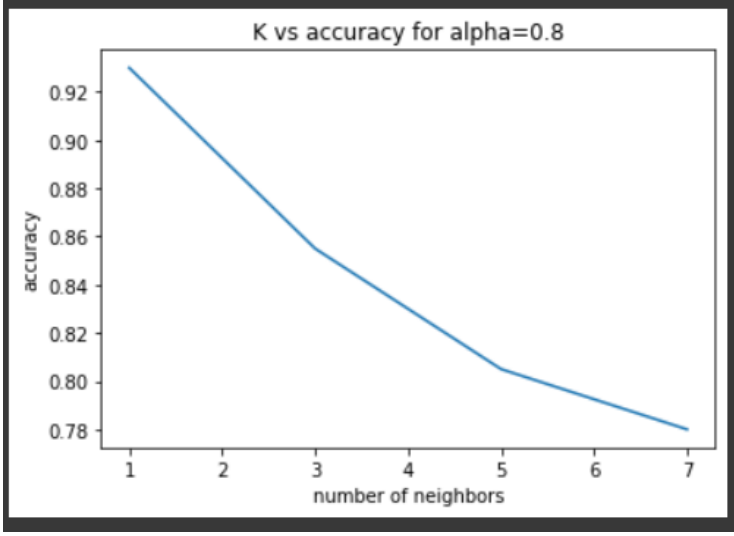
```
Accuracy with R =  37.0 :  0.93
Accuracy with R =  53.0 :  0.94
Accuracy with R =  77.0 :  0.945
Accuracy with R =  116.0 :  0.935
```

```
#k here is the number of neighbors
#the code loops over a different values of k then calculates the accuracy
#for each value of k for each of the 4 datasets
#this is more efficient than the previous block as it uses a loop to perform
#the same classification process with different values of k

#6.classifier tuning
k_values = [1, 3, 5, 7]

accuracy_R0=np.zeros(4)
accuracy_R1=np.zeros(4)
accuracy_R2=np.zeros(4)
accuracy_R3=np.zeros(4)

for i in range(0,4):
  knn = KNeighborsClassifier(n_neighbors=k_values[i])
  k=np.zeros(4)
  knn.fit(projected_data1.T, Ytrain)
  accuracy_R0[i]=knn.score(projected_data5.T, Ytest)
  print("Accuracy with R = ", R[0], "and number of neighbors =",k_values[i],": ", accuracy_R0[i])

  knn = KNeighborsClassifier(n_neighbors=k_values[i])
  knn.fit(projected_data2.T, Ytrain)
  accuracy_R1[i]=knn.score(projected_data6.T, Ytest)
  print("Accuracy with R = ", R[1],"and number of neighbors =",k_values[i], ": ",  accuracy_R1[i] )

  knn = KNeighborsClassifier(n_neighbors=k_values[i])
  knn.fit(projected_data3.T, Ytrain)
  accuracy_R2[i]=knn.score(projected_data7.T, Ytest)
  print("Accuracy with R = ", R[2],"and number of neighbors =",k_values[i], ": ", accuracy_R2[i])
```

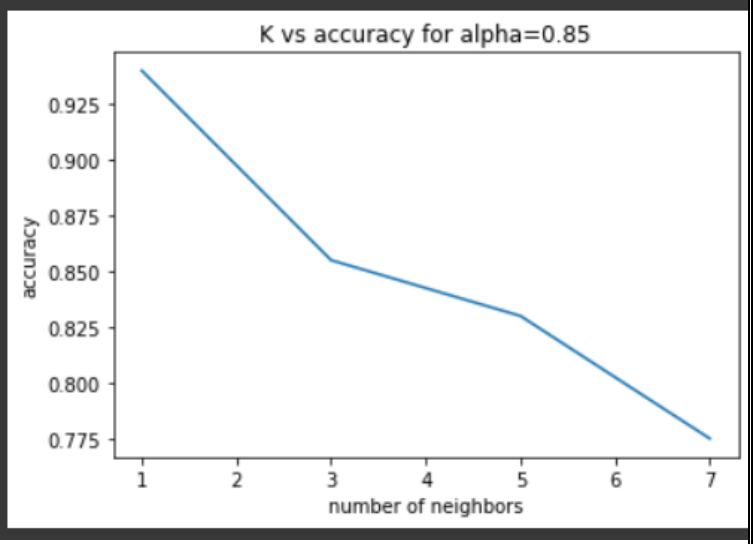
```
knn = KNeighborsClassifier(n_neighbors=k_values[i])
knn.fit(projected_data4.T, Ytrain)
accuracy_R3[i]=knn.score(projected_data8.T, Ytest)
print("Accuracy with R = ", R[3],"and number of neighbors =",k_values[i], ": ", accuracy_R3[i])
```

```
Accuracy with R =  37.0 and number of neighbors = 1 :  0.93
Accuracy with R =  53.0 and number of neighbors = 1 :  0.94
Accuracy with R =  77.0 and number of neighbors = 1 :  0.945
Accuracy with R =  116.0 and number of neighbors = 1 :  0.935
Accuracy with R =  37.0 and number of neighbors = 3 :  0.855
Accuracy with R =  53.0 and number of neighbors = 3 :  0.855
Accuracy with R =  77.0 and number of neighbors = 3 :  0.85
Accuracy with R =  116.0 and number of neighbors = 3 :  0.845
Accuracy with R =  37.0 and number of neighbors = 5 :  0.805
Accuracy with R =  53.0 and number of neighbors = 5 :  0.83
Accuracy with R =  77.0 and number of neighbors = 5 :  0.815
Accuracy with R =  116.0 and number of neighbors = 5 :  0.815
Accuracy with R =  37.0 and number of neighbors = 7 :  0.78
Accuracy with R =  53.0 and number of neighbors = 7 :  0.775
Accuracy with R =  77.0 and number of neighbors = 7 :  0.755
Accuracy with R =  116.0 and number of neighbors = 7 :  0.74
```

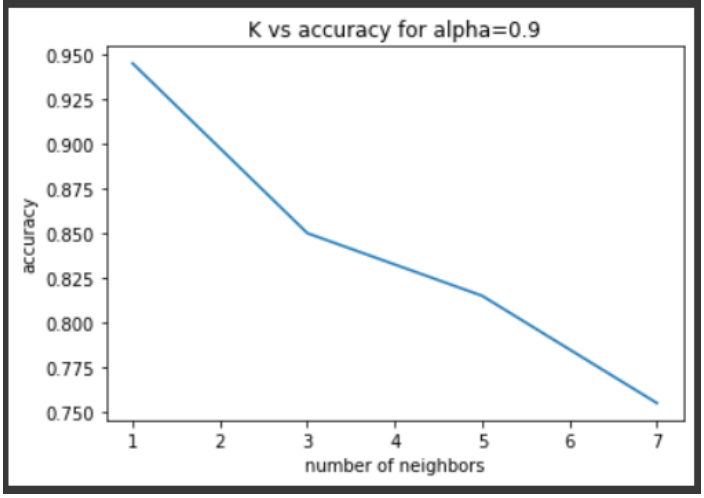
```
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R0)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('K vs accuracy for alpha=0.8')

# Show the plot
plt.show()
```



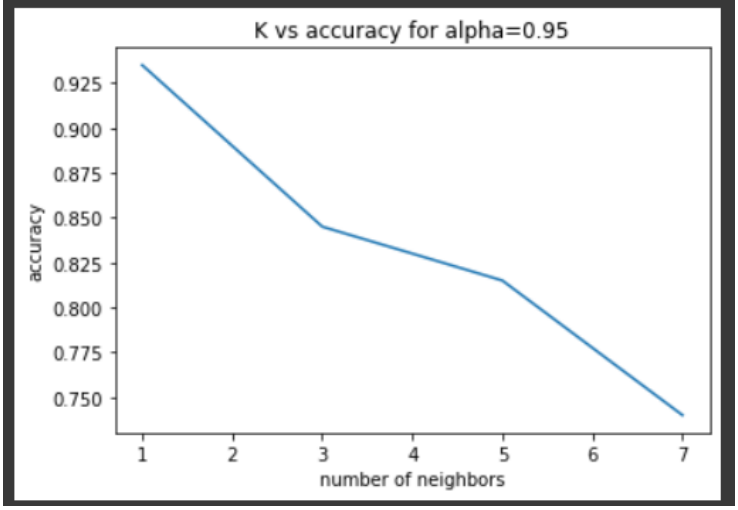

```
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R1)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('K vs accuracy for alpha=0.85')

# Show the plot
plt.show()
```



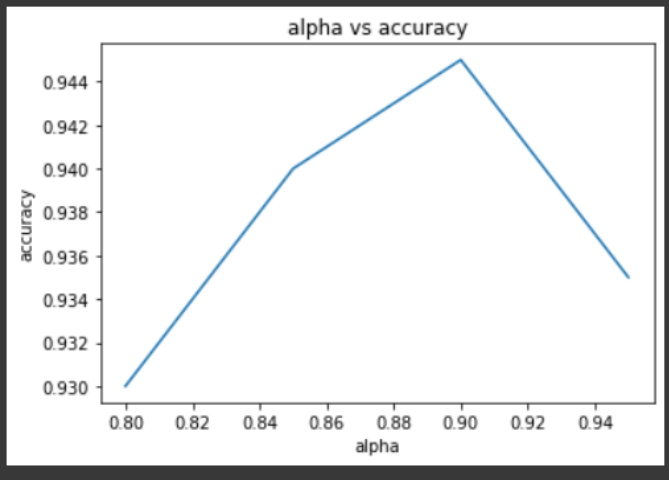
```
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R2)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('K vs accuracy for alpha=0.9')

# Show the plot
plt.show()
```




```
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R3)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('K vs accuracy for alpha=0.95')

# Show the plot
plt.show()
```




```
import matplotlib.pyplot as plt
plt.plot([0.8,0.85,0.9,0.95], [accuracy_R0[0],accuracy_R1[0],accuracy_R2[0],accuracy_R3[0]])

# Set axis labels and title
plt.xlabel('alpha')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy')

# Show the plot
plt.show()
```



Step 5,6:

5. LDA Classification

```
#5
#compute mean face for each of 40 individuals in the dataset
Mu = np.zeros((40, 10304))
for i in range (0,40):
  Mu[i]=np.mean(Dtrain[(i*5):(i*5)+4,:], axis=0)
print(Mu[1].reshape(10304,1).shape)
```

```
(10304, 1)
```

```
#compute the between class scatter matrix for 40 individuals
Sb=np.zeros((10304,10304))
for i in range (0,40):
  Sb=Sb+5*np.dot((Mu[i].reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)),(Mu[i].reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)).T)
print(Sb.shape)
```

```
(10304, 10304)
```

```
#Compute centered image data for each individual
LDAcentertrain = np.zeros(((40,5,10304)))
for i in range(0,40):
  for j in range(0,5):
    LDAcentertrain[i,j]+=Dtrain[int(i*5+j)]-Mu[int(i)]
```

```
#compute scatter matrix using LDA
S = np.zeros(((10304,10304)))
for i in range(0, 40):
    S+= np.matmul(LDAcentertrain[i].T,LDAcentertrain[i])
```

```
#compute the inverse square matrix S
IS = np.linalg.inv(S)
#compute eigenvalues and eigenvectors
eigenvalues2, eigenvectors2 = np.linalg.eigh(np.dot(IS,Sb))
#sort the eigenvalues and eigenvectors descendingly
idx = np.argsort(-np.absolute(eigenvalues2))
eigenValues2 = eigenvalues2[idx]
eigenVectors2 = eigenvectors2[:,idx]
```

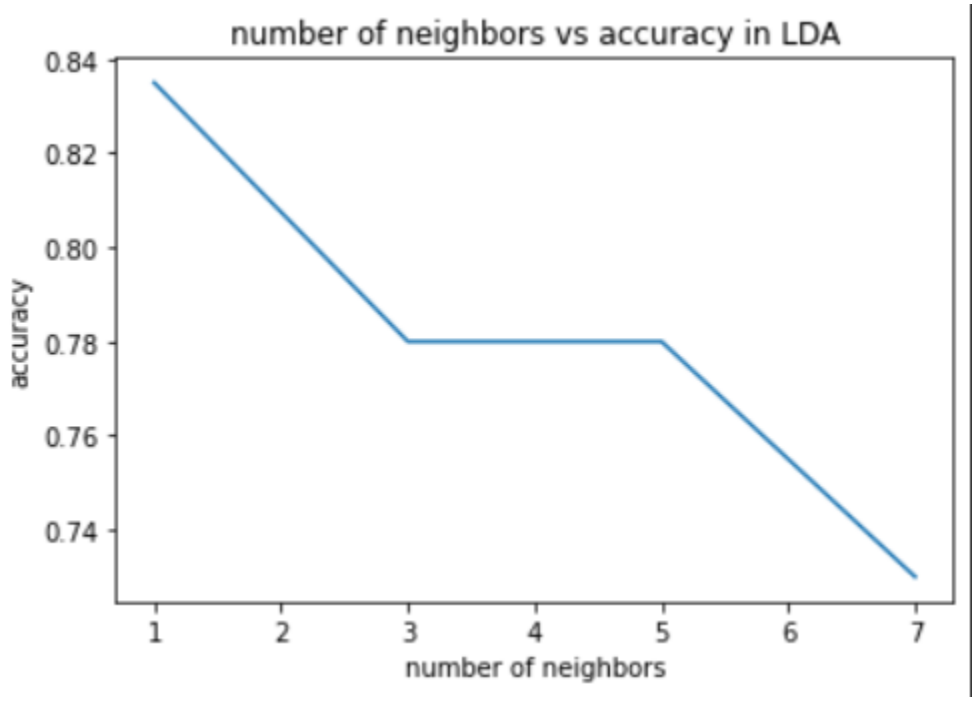
```
#perform dimensionality reduction on the training and test data
#by projecting them onto a subspace spanned by the top 39 eigenvectors
U5=eigenVectors2[:,:39]
projection_matrix = U5
projected_data9 = np.dot(projection_matrix.T, DTraincentered.T)
projected_data10 = np.dot(projection_matrix.T, DTestcentered.T)
#return the real part as new arrays
projected_data9_real = np.real(projected_data9.T)
projected_data10_real = np.real(projected_data10.T)
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
#train a k-nearest neighbors classifier and then evaluates the accuracy
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data9_real, Ytrain)
print("Accuracy with R = 39: ", knn.score(projected_data10_real, Ytest))
```

```
[ ]  #6.classifier tuning for LDA
     k_values = [1, 3, 5, 7]
     accuracy=np.zeros(4)

     for i in range (0,4):
       knn = KNeighborsClassifier(n_neighbors=k_values[i])
       knn.fit(projected_data9_real, Ytrain)
       accuracy[i]=knn.score(projected_data10_real, Ytest)
       print("Accuracy with number of neighbors ",k_values[i],": ", accuracy[i])
```

```
[ ]  import matplotlib.pyplot as plt
     plt.plot(k_values, accuracy)

     # Set axis labels and title
     plt.xlabel('number of neighbors')
     plt.ylabel('accuracy')
     plt.title('number of neighbors vs accuracy in LDA')

     # Show the plot
     plt.show()
```



Step 7:

7. Face Images VS Non-Face Images

```
[5]  parent_folder = '/content/drive/MyDrive/PR/natural_images'

     nonfacenum=700
     width = 92
     height = 112
     D2 = np.zeros((int(400+nonfacenum), 10304))
     DTraincentered2 = np.zeros((int(nonfacenum), 10304))
     DTestcentered2 = np.zeros((400, 10304))
     testc2=0
     trainc2=0
     Dtrain2 = np.zeros((int(nonfacenum), 10304))
     Dtest2 = np.zeros((400, 10304))
     Y2 = np.zeros((int(400+nonfacenum),1))
     Ytrain2 = np.zeros((int(nonfacenum), 1))
     Ytest2 = np.zeros((400, 1))
     D2[0:400]=D
     Y2=[np.ones((1,400)),np.zeros((1,int(nonfacenum)))]
     s=400
     # for sub in range(1,41):
     for filename in os.listdir(parent_folder):
       folder_path = f"{parent_folder}/{filename}"
       for i in range(0,100):
         if i<10:
           filename2 = f"{folder_path}/{filename}_000{i}.jpg"
         else:
           filename2 = f"{folder_path}/{filename}_00{i}.jpg"
         image = Image.open(filename2)
         image = image.resize((92, 112))
         image = image.convert('L')
```

```
[37]        image = image.convert('L')
            image_array = np.array(image)
            D2[s] = image_array.flatten()
            s=s+1

        np.random.shuffle(D2[0:400])
        np.random.shuffle(D2[400:int(nonfacenum+400)])
        Dtrain2[0:200]=D2[0:200]
        Dtrain2[200:int(nonfacenum)]=D2[400:int(nonfacenum)+200]
        Ytrain2[0:200]=np.ones((200,1))
        Dtest2[0:200]=D2[200:400]
        Dtest2[200:400]=D2[int(nonfacenum+200):int(nonfacenum+400)]
        Ytest2[0:200]=np.ones((200,1))

        accuracyPCA=np.zeros(3)
        accuracyLDA=np.zeros(3)
```

```
for i in range(0,10304):
  DTraincentered2[:,i]=Dtrain2[:,i]-Dtrain2[:,i].mean()

for i in range(0,10304):
  DTestcentered2[:,i]=Dtest2[:,i]-Dtrain2[:,i].mean()
```

```
[ ]  covarianceMatrix22 = np.cov(DTraincentered2.T)
     eigenvalues22, eigenvectors22 = np.linalg.eigh(covarianceMatrix22)
```

```
[ ]  idx = np.argsort(-(eigenvalues22))
     eigenValues22 = eigenvalues22[idx]
     eigenVectors22 = eigenvectors22[:,idx]
```

```
[9]  for i in range(0,int(nonfacenum)):
       explained=eigenValues22[0:i].sum()/eigenValues22.sum()
       if explained > 0.9:
         R=i
         break
     print(R)
```

```
     124
```

```
[10] U21=eigenVectors22[:,:int(R)]
     projection_matrix = U21
     projected_data21 = np.dot(projection_matrix.T, DTraincentered2.T)
     projected_data22 = np.dot(projection_matrix.T, DTestcentered2.T)
```

```
[12] knn = KNeighborsClassifier(n_neighbors=1)
     knn.fit(projected_data21.T, Ytrain2)

     accuracyPCA[2]= knn.score(projected_data22.T, Ytest2)
     print("Accuracy with R = ", R, ": ",accuracyPCA[2])
```

```
     Accuracy with R =  124 :  0.9525
     /usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py
       return self._fit(X, y)
```

```
[15] import os
     import cv2
     from sklearn import svm
     import numpy as np
     from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
     from sklearn.metrics import confusion_matrix
     # Train and evaluate classifier
     clf = svm.SVC()
     clf.fit(projected_data21.T, Ytrain2)
     test_pred = clf.predict(projected_data22.T)
     # Generate confusion matrix
     cm = confusion_matrix(Ytest2, test_pred)
     print("Confusion matrix:")
     print(cm)
```

```
     Confusion matrix:
     [[194   6]
      [  8 192]]
     /usr/local/lib/python3.9/dist-packages/sklearn/u
       y = column_or_1d(y, warn=True)
```

```
16]  Mu1=np.mean(Dtrain2[:200,:], axis=0)
     Mu2=np.mean(Dtrain2[200:,:], axis=0)
```

```
Sb=np.zeros((10304,10304))
Sb=200*np.dot((Mu1.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)),
              (Mu1.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)).T)+500*np.dot((Mu2.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1))
              ,(Mu2.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)).T)
print(Sb)
```

```
[[2099652.02   2113914.35   2075730.95   ... 2342922.72   2477439.1
  2567726.07  ]
 [2113914.35   2128280.053  2089835.225  ... 2359085.52   2494455.642
  2585353.153 ]
 [2075730.95   2089835.225  2052085.4375 ... 2316399.45   2449342.3375
  2538598.8   ]
 ...
 [2342922.72   2359085.52   2316399.45   ... 2623851.72   2771662.23
  2872304.94  ]
 [2477439.1    2494455.642  2449342.3375 ... 2771662.23   2928644.9005
  3035097.367 ]
 [2567726.07   2585353.153  2538598.8    ... 2872304.94   3035097.367
  3145433.423 ]]
```

```
LDAcentertrain22=np.zeros((nonfacenum,10304))
i=0
for r in Dtrain2:
  if i<200:
    LDAcentertrain22[i]=r-Mu1
  else:
    LDAcentertrain22[i]=r-Mu2
  i=i+1
```

```
S1=np.matmul(LDAcentertrain22[:200,:].T,LDAcentertrain22[:200,:])
S2=np.matmul(LDAcentertrain22[200:,:].T,LDAcentertrain22[200:,:])
S=S1+S2
```

```
IS = np.linalg.inv(S)
```

```
[22] eigenvalues22, eigenvectors22 = np.linalg.eigh(np.dot(IS,Sb))
```

```
[23] idx = np.argsort(-(eigenvalues22))
     eigenValues22 = eigenvalues22[idx]
     eigenVectors22 = eigenvectors22[:,idx]
```

```
[24] U22=eigenVectors22[:,:2]
     projection_matrix = U22
     projected_data22 = np.dot(projection_matrix.T, DTraincentered2.T)
     projected_data222 = np.dot(projection_matrix.T, DTestcentered2.T)
```

```
[25] knn = KNeighborsClassifier(n_neighbors=1)
     knn.fit(projected_data22.T, Ytrain2)
     accuracyLDA[2]=knn.score(projected_data222.T, Ytest2)
     print("Accuracy with R = 1: ",accuracyLDA[2] )
```

```
     Accuracy with R = 1:  0.8425
     /usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classific
       return self._fit(X, y)
```

```
import os
import cv2
from sklearn import svm
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
# Train and evaluate classifier
clf = svm.SVC()
clf.fit(projected_data22.T, Ytrain2)
test_pred = clf.predict(projected_data222.T)


# Generate confusion matrix
cm = confusion_matrix(Ytest2, test_pred)
print("Confusion matrix:")
print(cm)
```

```
Confusion matrix:
[[180  20]
 [ 15 185]]
/usr/local/lib/python3.9/dist-packages/sklearn/utils/validation.py:11
  y = column_or_1d(y, warn=True)
```

```
DTraincentered2 = np.zeros((400, 10304))
trainc2=0
Dtrain2 = np.zeros((400, 10304))
Ytrain2 = np.zeros((400, 1))
Dtrain2[0:200]=D2[0:200]
Dtrain2[200:400]=D2[400:400+200]
Ytrain2[0:200]=np.ones((200,1))
Ytrain2[200:400]=np.zeros((200,1))
```

```
for i in range(0,10304):
  DTraincentered2[:,i]=Dtrain2[:,i]-Dtrain2[:,i].mean()

for i in range(0,10304):
  DTestcentered2[:,i]=Dtest2[:,i]-Dtrain2[:,i].mean()
```

```
covarianceMatrix22 = np.cov(DTraincentered2.T)
eigenvalues22, eigenvectors22 = np.linalg.eigh(covarianceMatrix22)
```

```
idx = np.argsort(-(eigenvalues22))
eigenValues22 = eigenvalues22[idx]
eigenVectors22 = eigenvectors22[:,idx]

for i in range(0,400):
  explained=eigenValues22[0:i].sum()/eigenValues22.sum()
  if explained > 0.9:
    R=i
    break
print(R)
```

```
U21=eigenVectors22[:,:int(R)]
projection_matrix = U21
projected_data21 = np.dot(projection_matrix.T, DTraincentered2.T)
projected_data22 = np.dot(projection_matrix.T, DTestcentered2.T)

projected_data21 = np.real(projected_data21.T)
projected_data22 = np.real(projected_data22.T)


knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data21, Ytrain2)

accuracyPCA[0]=knn.score(projected_data22, Ytest2)
print("Accuracy with R = ", R, ": ",accuracyPCA[0] )
```

```
93
Accuracy with R =  93 :  0.92
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classificati
  return self._fit(X, y)
```

```
import os
import cv2
from sklearn import svm
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
# Train and evaluate classifier
clf = svm.SVC()
clf.fit(projected_data21, Ytrain2)
test_pred = clf.predict(projected_data22)
# Generate confusion matrix
cm = confusion_matrix(Ytest2, test_pred)
print("Confusion matrix:")
print(cm)
```

```
Confusion matrix:
[[197   3]
 [  2 198]]
/usr/local/lib/python3.9/dist-packages/sklearn/utils/validation.py:1143: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Pl
  y = column_or_1d(y, warn=True)
```

```
Mu1=np.mean(Dtrain2[:200,:], axis=0)
Mu2=np.mean(Dtrain2[200:,:], axis=0)
```

```
Sb=np.zeros((10304,10304))
Sb=200*np.dot((Mu1.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)),
              (Mu1.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)).T)+500*np.dot((Mu2.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)),
              (Mu2.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)).T)
print(Sb)
```

```
[[2109611.33   2119304.49   2075878.625  ... 2290126.65   2405745.775
  2474726.31  ]
 [2119304.49   2129045.663  2085420.907  ... 2300548.052  2416700.718
  2486004.29  ]
 [2075878.625  2085420.907  2042690.1805 ... 2253387.673  2367160.7695
  2435045.515 ]
 ...
 [2290126.65   2300548.052  2253387.673  ... 2489034.278  2614479.777
  2689185.59  ]
 [2405745.775  2416700.718  2367160.7695 ... 2614479.777  2746263.4055
  2824753.785 ]
 [2474726.31   2486004.29   2435045.515  ... 2689185.59   2824753.785
  2905510.37  ]]
```

```
LDAcentertrain22=np.zeros((nonfacenum,10304))
i=0
for r in Dtrain2:
  if i<200:
    LDAcentertrain22[i]=r-Mu1
  else:
    LDAcentertrain22[i]=r-Mu2
  i=i+1
```

+ Code + Text

```
[17] S1=np.matmul(LDAcentertrain22[:200,:].T,LDAcentertrain22[:200,:])
     S2=np.matmul(LDAcentertrain22[200:,:].T,LDAcentertrain22[200:,:])
     S=S1+S2
```

```
[18] IS = np.linalg.inv(S)
```

```
[19] eigenvalues22, eigenvectors22 = np.linalg.eigh(np.dot(IS,Sb))
```

```
[20] idx = np.argsort(-(eigenvalues22))
     eigenValues22 = eigenvalues22[idx]
     eigenVectors22 = eigenvectors22[:,idx]
```

```
[21] U22=eigenVectors22[:,:2]
     projection_matrix = U22
     projected_data22 = np.dot(projection_matrix.T, DTraincentered2.T)
     projected_data222 = np.dot(projection_matrix.T, DTestcentered2.T)
```

```
[22] knn = KNeighborsClassifier(n_neighbors=1)
     knn.fit(projected_data22.T, Ytrain2)
     accuracyLDA[2]=knn.score(projected_data222.T, Ytest2)
     print("Accuracy with R = 1: ",accuracyLDA[2] )
```

```
Accuracy with R = 1:  0.84
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classificatio
  return self._fit(X, y)
```

```
[24] import os
     import cv2
     from sklearn import svm
     import numpy as np
     from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
     from sklearn.metrics import confusion_matrix
     # Train and evaluate classifier
     clf = svm.SVC()
     clf.fit(projected_data22.T, Ytrain2)
     test_pred = clf.predict(projected_data222.T)


     # Generate confusion matrix
     cm = confusion_matrix(Ytest2, test_pred)
     print("Confusion matrix:")
     print(cm)
```

```
Confusion matrix:
[[176  24]
 [ 17 183]]
```

```
DTraincentered2 = np.zeros((400, 10304))

trainc2=0
Dtrain2 = np.zeros((400, 10304))

Ytrain2 = np.zeros((400, 1))


Dtrain2[0:200]=D2[0:200]
Dtrain2[200:400]=D2[400:400+200]
Ytrain2[0:200]=np.ones((200,1))
Ytrain2[200:400]=np.zeros((200,1))

for i in range(0,10304):
  DTraincentered2[:,i]=Dtrain2[:,i]-Dtrain2[:,i].mean()

for i in range(0,10304):
  DTestcentered2[:,i]=Dtest2[:,i]-Dtrain2[:,i].mean()
```

```
covarianceMatrix22 = np.cov(DTraincentered2.T)
eigenvalues22, eigenvectors22 = np.linalg.eigh(covarianceMatrix22)
```

```
idx = np.argsort(-(eigenvalues22))
eigenValues22 = eigenvalues22[idx]
eigenVectors22 = eigenvectors22[:,idx]

for i in range(0,400):
  explained=eigenValues22[0:i].sum()/eigenValues22.sum()
  if explained > 0.9:
    R=i
    break
print(R)

U21=eigenVectors22[:,:int(R)]
projection_matrix = U21
projected_data21 = np.dot(projection_matrix.T, DTraincentered2.T)
projected_data22 = np.dot(projection_matrix.T, DTestcentered2.T)

projected_data21 = np.real(projected_data21.T)
projected_data22 = np.real(projected_data22.T)


knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data21, Ytrain2)

accuracyPCA[0]=knn.score(projected_data22, Ytest2)
print("Accuracy with R = ", R, ": ",accuracyPCA[0] )
```

```
90
Accuracy with R =  90 :  0.885
```

```
import os
import cv2
from sklearn import svm
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
# Train and evaluate classifier
clf = svm.SVC()
clf.fit(projected_data21, Ytrain2)
test_pred = clf.predict(projected_data22)

# Generate confusion matrix
cm = confusion_matrix(Ytest2, test_pred)
print("Confusion matrix:")
print(cm)
```

```
Confusion matrix:
[[188  12]
 [  0 200]]
```

```
Mu1=np.mean(Dtrain2[:200,:], axis=0)
Mu2=np.mean(Dtrain2[200:,:], axis=0)

Sb=np.zeros((10304,10304))
Sb=200*np.dot((Mu1.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)),
              (Mu1.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)).T)+200*np.dot((Mu2.reshape(10304,1)
              -np.mean(Dtrain, axis=0).reshape(10304,1)),(Mu2.reshape(10304,1)-np.mean(Dtrain, axis=0).reshape(10304,1)).T)
LDAcentertrain22=np.zeros((400,10304))
i=0
for r in Dtrain2:
  if i<200:
    LDAcentertrain22[i]=r-Mu1
  else:
    LDAcentertrain22[i]=r-Mu2
  i=i+1

S1=np.matmul(LDAcentertrain22[:200,:].T,LDAcentertrain22[:200,:])
S2=np.matmul(LDAcentertrain22[200:,:].T,LDAcentertrain22[200:,:])
S=S1+S2
IS = np.linalg.inv(S)
```

```
[10] eigenvalues22, eigenvectors22 = np.linalg.eigh(np.dot(IS,Sb))
```

```
[11] idx = np.argsort(-(eigenvalues22))
     eigenValues22 = eigenvalues22[idx]
     eigenVectors22 = eigenvectors22[:,idx]

     U22=eigenVectors22[:,:2]
     projection_matrix = U22
     projected_data22 = np.dot(projection_matrix.T, DTraincentered2.T)
     projected_data222 = np.dot(projection_matrix.T, DTestcentered2.T)
```

```
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data22.T, Ytrain2)

accuracyLDA[0]=knn.score(projected_data222.T, Ytest2)
print("Accuracy with R = 1: ",accuracyLDA[0] )
```

```
Accuracy with R = 1:  0.905
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py:215: DataC
  return self._fit(X, y)
```

```
[14] import os
     import cv2
     from sklearn import svm
     import numpy as np
     from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
     from sklearn.metrics import confusion_matrix
     # Train and evaluate classifier
     clf = svm.SVC()
     clf.fit(projected_data22.T, Ytrain2)
     test_pred = clf.predict(projected_data222.T)


     # Generate confusion matrix
     cm = confusion_matrix(Ytest2, test_pred)
     print("Confusion matrix:")
     print(cm)
```

```
Confusion matrix:
[[160  40]
 [  7 193]]
```

```
DTraincentered2 = np.zeros((600, 10304))
trainc2=0
Dtrain2 = np.zeros((600, 10304))
Ytrain2 = np.zeros((600, 1))
Dtrain2[0:200]=D2[0:200]
Dtrain2[200:600]=D2[400:800]
Ytrain2[0:200]=np.ones((200,1))
Ytrain2[200:600]=np.zeros((400,1))

for i in range(0,10304):
  DTraincentered2[:,i]=Dtrain2[:,i]-Dtrain2[:,i].mean()

for i in range(0,10304):
  DTestcentered2[:,i]=Dtest2[:,i]-Dtrain2[:,i].mean()
```

```
[16] covarianceMatrix22 = np.cov(DTraincentered2.T)
     eigenvalues22, eigenvectors22 = np.linalg.eigh(covarianceMatrix22)
```

```
[17] idx = np.argsort(-(eigenvalues22))
     eigenValues22 = eigenvalues22[idx]
     eigenVectors22 = eigenvectors22[:,idx]

     for i in range(0,600):
       explained=eigenValues22[0:i].sum()/eigenValues22.sum()
       if explained > 0.9:
         R=i
         break
     print(R)

     U21=eigenVectors22[:,:int(R)]
     projection_matrix = U21
     projected_data21 = np.dot(projection_matrix.T, DTraincentered2.T)
     projected_data22 = np.dot(projection_matrix.T, DTestcentered2.T)
```

```
    i=0
    for r in Dtrain2:
      if i<200:
        LDAcentertrain22[i]=r-Mu1
      else:
        LDAcentertrain22[i]=r-Mu2
      i=i+1

    S1=np.matmul(LDAcentertrain22[:200,:].T,LDAcentertrain22[:200,:])
    S2=np.matmul(LDAcentertrain22[200:,:].T,LDAcentertrain22[200:,:])
    S=S1+S2
    IS = np.linalg.inv(S)
```

```
[15] eigenvalues22, eigenvectors22 = np.linalg.eigh(np.dot(IS,Sb))
```

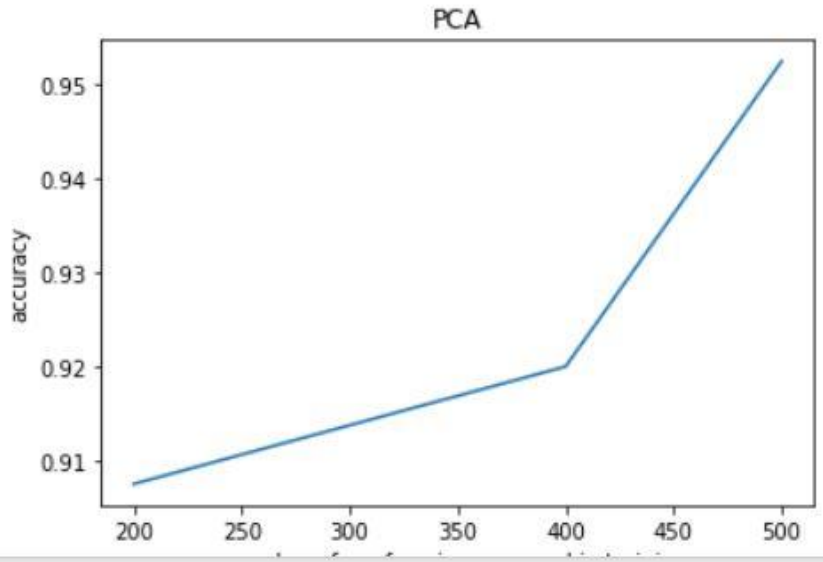
```
[16] idx = np.argsort(-(eigenvalues22))
     eigenValues22 = eigenvalues22[idx]
     eigenVectors22 = eigenvectors22[:,idx]

     U22=eigenVectors22[:,:2]
     projection_matrix = U22
     projected_data22 = np.dot(projection_matrix.T, DTraincentered2.T)
     projected_data222 = np.dot(projection_matrix.T, DTestcentered2.T)

     knn = KNeighborsClassifier(n_neighbors=1)
     knn.fit(projected_data22.T, Ytrain2)

     accuracyLDA[1]=knn.score(projected_data222.T, Ytest2)
     print("Accuracy with R = 1: ",accuracyLDA[1] )
```

```
     Accuracy with R = 1:  0.84
```

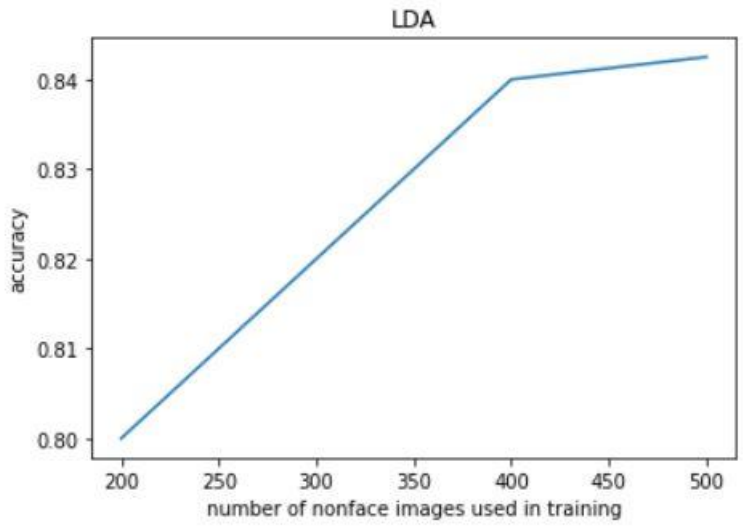
```
import os
import cv2
from sklearn import svm
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
# Train and evaluate classifier
clf = svm.SVC()
clf.fit(projected_data22.T, Ytrain2)
test_pred = clf.predict(projected_data222.T)


# Generate confusion matrix
cm = confusion_matrix(Ytest2, test_pred)
print("Confusion matrix:")
print(cm)
```

```
Confusion matrix:
[[180  20]
 [ 21 179]]
```

```
import matplotlib.pyplot as plt


n=np.array([200,400,500])
plt.plot(n, accuracyPCA)
# Set axis labels and title
plt.xlabel('number of nonface images used in training')
plt.ylabel('accuracy')
plt.title('PCA')

# Show the plot
plt.show()
```



```
import matplotlib.pyplot as plt
plt.plot(n, accuracyLDA)

# Set axis labels and title
plt.xlabel('number of nonface images used in training')
plt.ylabel('accuracy')
plt.title('LDA')

# Show the plot
plt.show()
```



Step 8 (Bonus):

```python
#Bonus Part1
from PIL import Image
import numpy as np
import os

parent_folder = '/content/drive/MyDrive/PR/assignment1'
width = 92
height = 112
D = np.zeros((400, 10304))
DTraincentered73 = np.zeros((280, 10304))
DTestcentered73 = np.zeros((120, 10304))
testc = 0
trainc = 0
Dtrain73 = np.zeros((280, 10304))
Dtest73 = np.zeros((120, 10304))
Y = np.zeros(400)
Ytrain73 = np.zeros((280, 1))
Ytest73 = np.zeros((120, 1))

for sub in range(1, 41):
    folder_path = f"{parent_folder}/s{sub}"
    sub_data = np.zeros((10, 10304))
    for i in range(1, 11):
        filename = f"{folder_path}/{i}.pgm"
        image = Image.open(filename)
        image_array = np.array(image)
        sub_data[(i - 1), :] = image_array.flatten()

    np.random.shuffle(sub_data)
    train_data = sub_data[:7, :]
    test_data = sub_data[7:, :]
    #7 instances for train w 3 for test
    Dtrain73[trainc:trainc+7, :] = train_data
    Dtest73[testc:testc+3, :] = test_data
    Ytrain73[trainc:trainc+7, 0] = sub
    Ytest73[testc:testc+3, 0] = sub
```

```
    trainc += 7
    testc += 3


DTraincentered73 = Dtrain73 - np.mean(Dtrain73, axis=0)
DTestcentered73 = Dtest73 - np.mean(Dtrain73, axis=0)
```

```
#using part 1 in the default PCA algorithm
covarianceMatrix73 = np.cov(DTraincentered73.T)

eigenvalues73, eigenvectors73 = np.linalg.eigh(covarianceMatrix73)

idx = np.argsort(-np.absolute(eigenvalues73))
eigenValues73 = eigenvalues73[idx]
eigenVectors73 = eigenvectors73[:,idx]
```

```
R=np.zeros(4)
for i in range(0,10304):
  explained=eigenValues73[0:i].sum()/eigenValues73.sum()
  if explained > 0.8:
    R[0]=i
    break
for i in range(i,10304):
  explained=eigenValues73[0:i].sum()/eigenValues73.sum()
  if explained > 0.85:
    R[1]=i
    break
for i in range(i,10304):
  explained=eigenValues73[0:i].sum()/eigenValues73.sum()
  if explained > 0.9:
    R[2]=i
    break
for i in range(i,10304):
  explained=eigenValues73[0:i].sum()/eigenValues73.sum()
  if explained > 0.95:
    R[3]=i
    break
print(R)
```

```
[ 40.  60.  92. 148.]
```

```
U1_73=eigenVectors73[:,:int(R[0])]
U2_73=eigenVectors73[:,:int(R[1])]
U3_73=eigenVectors73[:,:int(R[2])]
U4_73=eigenVectors73[:,:int(R[3])]

projection_matrix73 = U1_73
projected_data173 = np.dot(projection_matrix73.T, DTraincentered73.T)
projection_matrix73 = U2_73
projected_data273 = np.dot(projection_matrix73.T, DTraincentered73.T)
projection_matrix73 = U3_73
projected_data373 = np.dot(projection_matrix73.T, DTraincentered73.T)
projection_matrix73 = U4_73
projected_data473 = np.dot(projection_matrix73.T, DTraincentered73.T)

projection_matrix73 = U1_73
projected_data573 = np.dot(projection_matrix73.T, DTestcentered73.T)
projection_matrix73 = U2_73
projected_data673 = np.dot(projection_matrix73.T, DTestcentered73.T)
projection_matrix73 = U3_73
projected_data773 = np.dot(projection_matrix73.T, DTestcentered73.T)
projection_matrix73 = U4_73
projected_data873 = np.dot(projection_matrix73.T, DTestcentered73.T)
```

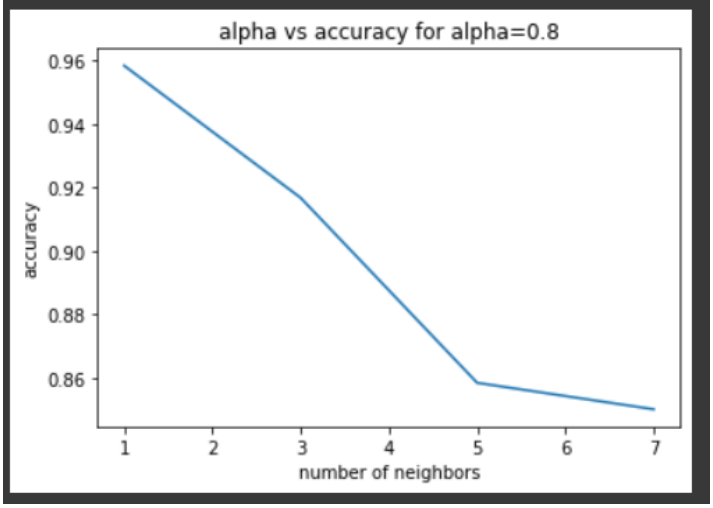
```
from sklearn.neighbors import KNeighborsClassifier
knn73 = KNeighborsClassifier(n_neighbors=1)
knn73.fit(projected_data173.T, Ytrain73)
print("Accuracy with R = ", R[0], ": ", knn73.score(projected_data573.T, Ytest73))

knn73 = KNeighborsClassifier(n_neighbors=1)
knn73.fit(projected_data273.T, Ytrain73)
print("Accuracy with R = ", R[1], ": ", knn73.score(projected_data673.T, Ytest73))

knn73 = KNeighborsClassifier(n_neighbors=1)
knn73.fit(projected_data373.T, Ytrain73)
print("Accuracy with R = ", R[2], ": ", knn73.score(projected_data773.T, Ytest73))

knn73 = KNeighborsClassifier(n_neighbors=1)
knn73.fit(projected_data473.T, Ytrain73)
print("Accuracy with R = ", R[3], ": ", knn73.score(projected_data873.T, Ytest73))
```

```
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py:215: Da
  return self._fit(X, y)
Accuracy with R =  40.0 :  0.9583333333333334
Accuracy with R =  60.0 :  0.9583333333333334
Accuracy with R =  92.0 :  0.9583333333333334
Accuracy with R =  148.0 :  0.9583333333333334
```

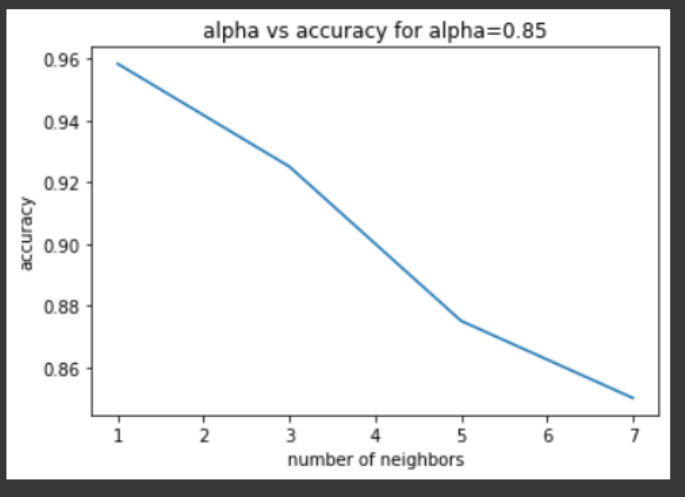
```
k_values = [1, 3, 5, 7]

accuracy_R073=np.zeros(4)
accuracy_R173=np.zeros(4)
accuracy_R273=np.zeros(4)
accuracy_R373=np.zeros(4)

for i in range(0,4):
  knn73 = KNeighborsClassifier(n_neighbors=k_values[i])
  k73=np.zeros(4)
  knn73.fit(projected_data173.T, Ytrain73)
  accuracy_R073[i]=knn73.score(projected_data573.T, Ytest73)
  print("Accuracy with R = ", R[0], "and number of neighbors =",k_values[i],": ", accuracy_R073[i])

  knn73 = KNeighborsClassifier(n_neighbors=k_values[i])
  knn73.fit(projected_data273.T, Ytrain73)
  accuracy_R173[i]=knn73.score(projected_data673.T, Ytest73)
  print("Accuracy with R = ", R[1],"and number of neighbors =",k_values[i], ": ",  accuracy_R173[i] )

  knn73 = KNeighborsClassifier(n_neighbors=k_values[i])
  knn73.fit(projected_data373.T, Ytrain73)
  accuracy_R273[i]=knn73.score(projected_data773.T, Ytest73)
  print("Accuracy with R = ", R[2],"and number of neighbors =",k_values[i], ": ", accuracy_R273[i])

  knn73 = KNeighborsClassifier(n_neighbors=k_values[i])
  knn73.fit(projected_data473.T, Ytrain73)
  accuracy_R373[i]=knn73.score(projected_data873.T, Ytest73)
  print("Accuracy with R = ", R[3],"and number of neighbors =",k_values[i], ": ", accuracy_R373[i])
```

```
Accuracy with R =  40.0 and number of neighbors = 1 :  0.9583333333333334
Accuracy with R =  60.0 and number of neighbors = 1 :  0.9583333333333334
Accuracy with R =  92.0 and number of neighbors = 1 :  0.9583333333333334
Accuracy with R =  148.0 and number of neighbors = 1 :  0.9583333333333334
Accuracy with R =  40.0 and number of neighbors = 3 :  0.9166666666666666
Accuracy with R =  60.0 and number of neighbors = 3 :  0.925
Accuracy with R =  92.0 and number of neighbors = 3 :  0.9166666666666666
Accuracy with R =  148.0 and number of neighbors = 3 :  0.9083333333333333
Accuracy with R =  40.0 and number of neighbors = 5 :  0.8583333333333333
Accuracy with R =  60.0 and number of neighbors = 5 :  0.875
Accuracy with R =  92.0 and number of neighbors = 5 :  0.8833333333333333
Accuracy with R =  148.0 and number of neighbors = 5 :  0.875
Accuracy with R =  40.0 and number of neighbors = 7 :  0.85
Accuracy with R =  60.0 and number of neighbors = 7 :  0.85
Accuracy with R =  92.0 and number of neighbors = 7 :  0.8
Accuracy with R =  148.0 and number of neighbors = 7 :  0.8
```

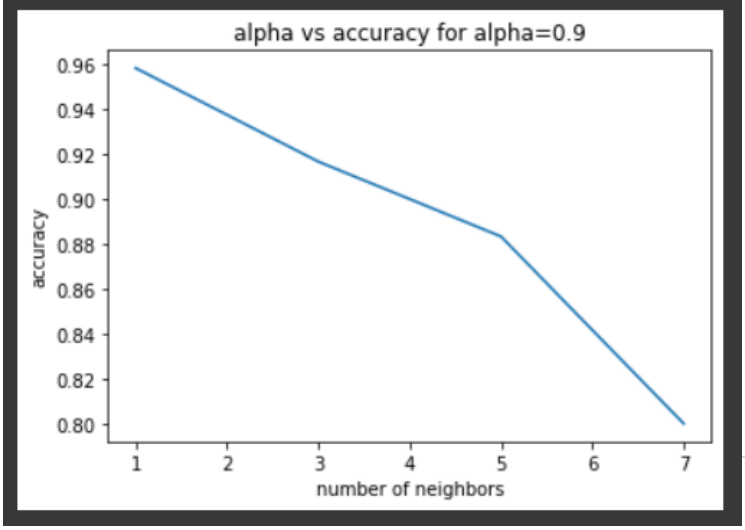
```
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R073)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy for alpha=0.8')

# Show the plot
plt.show()
```



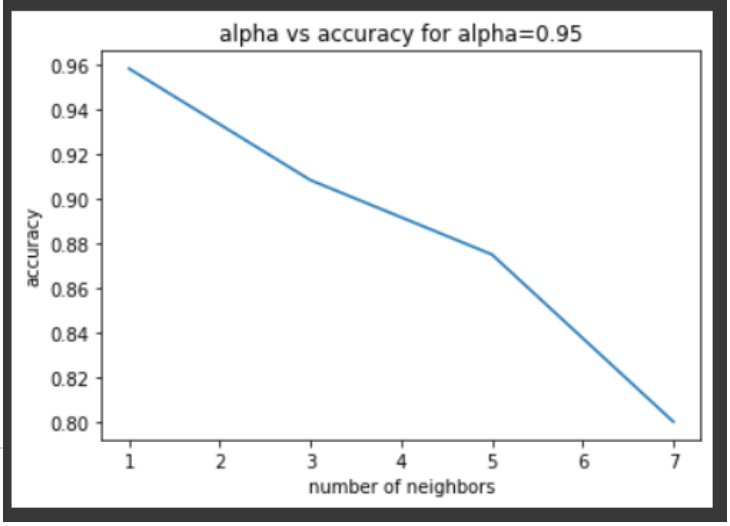

```
[10] import matplotlib.pyplot as plt
     plt.plot(k_values,accuracy_R173)

     # Set axis labels and title
     plt.xlabel('number of neighbors')
     plt.ylabel('accuracy')
     plt.title('alpha vs accuracy for alpha=0.85')

     # Show the plot
     plt.show()
```



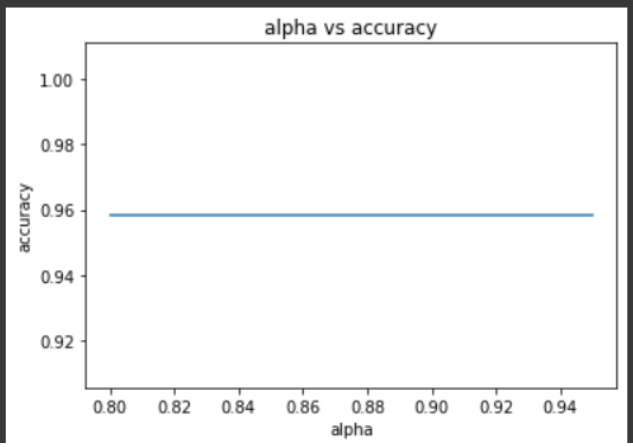

```
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R273)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy for alpha=0.9')

# Show the plot
plt.show()
```




```
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R373)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy for alpha=0.95')

# Show the plot
plt.show()
```



```
import matplotlib.pyplot as plt
plt.plot([0.8,0.85,0.9,0.95], [accuracy_R073[0],accuracy_R173[0],accuracy_R273[0],accuracy_R373[0]])

# Set axis labels and title
plt.xlabel('alpha')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy')

# Show the plot
plt.show()
```




```
#LDA 70 30
#compute mean face for each of 40 individuals in the dataset
Mu73 = np.zeros((40, 10304))
for i in range (0,40):
  Mu73[i]=np.mean(Dtrain73[(i*5):(i*5)+4,:], axis=0)
print(Mu73[1].reshape(10304,1).shape)
```

```
(10304, 1)
```

```
#compute the between class scatter matrix for 40 individuals
Sb73=np.zeros((10304,10304))
for i in range (0,40):
  Sb73=Sb73+5*np.dot((Mu73[i].reshape(10304,1)-np.mean(Dtrain73, axis=0).reshape(10304,1)),
                     (Mu73[i].reshape(10304,1)-np.mean(Dtrain73, axis=0).reshape(10304,1)).T)
print(Sb73.shape)
```

```
(10304, 10304)
```

```
[31] #Compute centered image data for each individual
     LDAcentertrain73 = np.zeros(((40,5,10304)))
     for i in range(0,40):
       for j in range(0,5):
         LDAcentertrain73[i,j]+=Dtrain73[int(i*5+j)]-Mu73[int(i)]
```

```
[32] #compute scatter matrix using LDA
     S73 = np.zeros(((10304,10304)))
     for i in range(0, 40):
         S73+= np.matmul(LDAcentertrain73[i].T,LDAcentertrain73[i])
```

```
[18] #compute the inverse square matrix S
     IS73 = np.linalg.inv(S73)
     #compute eigenvalues and eigenvectors
     eigenvalues273, eigenvectors273 = np.linalg.eigh(np.dot(IS73,Sb73))
     #sort the eigenvalues and eigenvectors descendingly
     idx73 = np.argsort(-np.absolute(eigenvalues273))
     eigenValues273 = eigenvalues273[idx73]
     eigenVectors273 = eigenvectors273[:,idx73]
```

```
[19] #perform dimensionality reduction on the training and test data
     #by projecting them onto a subspace spanned by the top 39 eigenvectors
     U573=eigenVectors273[:,:39]
     projection_matrix73 = U573
     projected_data973 = np.dot(projection_matrix73.T, DTraincentered73.T)
     projected_data1073 = np.dot(projection_matrix73.T, DTestcentered73.T)
```

```
from sklearn.neighbors import KNeighborsClassifier
#train a k-nearest neighbors classifier and then evaluates the accuracy
knn73 = KNeighborsClassifier(n_neighbors=1)
knn73.fit(projected_data973.T, Ytrain73)
print("Accuracy with R = 39: ", knn73.score(projected_data1073.T, Ytest73))
```

```
Accuracy with R = 39:  0.95
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py:1
  return self._fit(X, y)
```

Part Two PCA:

```
#Bonus Part2 Incremental PCA
from PIL import Image
import numpy as np
import os
from sklearn.neighbors import KNeighborsClassifier
from sklearn.decomposition import IncrementalPCA

parent_folder = '/content/drive/MyDrive/PR/assignment1'

width = 92
height = 112
D = np.zeros((400, 10304))
DTraincentered = np.zeros((200, 10304))
DTestcentered = np.zeros((200, 10304))
testc=0
trainc=0
Dtrain = np.zeros((200, 10304))
Dtest = np.zeros((200, 10304))
Y = np.zeros(400)
Ytrain = np.zeros((200, 1))
Ytest = np.zeros((200, 1))
```

```
for sub in range(1,41):
  folder_path = f"{parent_folder}/s{sub}"
  for i in range(1,11):
    filename = f"{folder_path}/{i}.pgm"
    image = Image.open(filename)
    image_array = np.array(image)
    D[(sub-1)*10+(i-1)] = image_array.flatten()
    Y[(sub-1)*10+(i-1)] = sub
    if (i % 2) != 0:
      Dtest[testc] = D[(sub-1)*10+(i-1)]
      Ytest[int(testc)] = Y[(sub-1)*10+(i-1)]
      testc=testc+1
    else:
      Dtrain[trainc] = D[(sub-1)*10+(i-1)]
      Ytrain[int(trainc)] = Y[(sub-1)*10+(i-1)]
      trainc=trainc+1

for i in range(0,10304):
  DTraincentered[:,i]=Dtrain[:,i]-Dtrain[:,i].mean()

for i in range(0,10304):
  DTestcentered[:,i]=Dtest[:,i]-Dtrain[:,i].mean()

n_components = 200
batch_size = 50

ipca = IncrementalPCA(n_components=n_components, batch_size=batch_size)
ipca.fit(DTraincentered)
eigenValues = ipca.explained_variance_
eigenVectors = ipca.components_.T

idx = np.argsort(-np.absolute(eigenValues))
eigenVal = eigenValues[idx]
eigenVec = eigenVectors[:,idx]
```

```
R=np.zeros(4)
for i in range(0,10304):
  explained=eigenVal[0:i].sum()/eigenVal.sum()
  if explained > 0.8:
    R[0]=i
    break
for i in range(i,10304):
  explained=eigenVal[0:i].sum()/eigenVal.sum()
  if explained > 0.85:
    R[1]=i
    break
for i in range(i,10304):
  explained=eigenVal[0:i].sum()/eigenVal.sum()
  if explained > 0.9:
    R[2]=i
    break
for i in range(i,10304):
  explained=eigenVal[0:i].sum()/eigenVal.sum()
  if explained > 0.95:
    R[3]=i
    break
print(R)
```

```
[ 37.  53.  77. 116.]
```

```
U1=eigenVec[:,:int(R[0])]
U2=eigenVec[:,:int(R[1])]
U3=eigenVec[:,:int(R[2])]
U4=eigenVec[:,:int(R[3])]
```

```
projection_matrix = U1
projected_data1_real = np.dot(projection_matrix.T, DTraincentered.T)
projection_matrix = U2
projected_data2_real = np.dot(projection_matrix.T, DTraincentered.T)
projection_matrix = U3
projected_data3_real = np.dot(projection_matrix.T, DTraincentered.T)
projection_matrix = U4
projected_data4_real = np.dot(projection_matrix.T, DTraincentered.T)

projection_matrix = U1
projected_data5_real = np.dot(projection_matrix.T, DTestcentered.T)
projection_matrix = U2
projected_data6_real = np.dot(projection_matrix.T, DTestcentered.T)
projection_matrix = U3
projected_data7_real = np.dot(projection_matrix.T, DTestcentered.T)
projection_matrix = U4
projected_data8_real = np.dot(projection_matrix.T, DTestcentered.T)
```

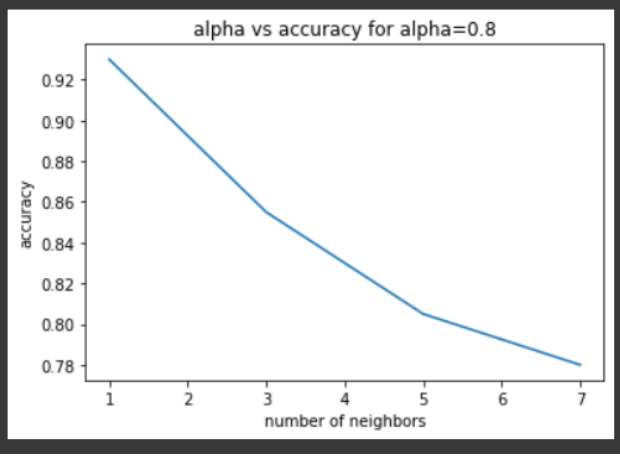
```
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data1_real.T, Ytrain)
print("Accuracy with R = ", R[0], ": ", knn.score(projected_data5_real.T, Ytest))

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data2_real.T, Ytrain)
print("Accuracy with R = ", R[1], ": ", knn.score(projected_data6_real.T, Ytest))

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data3_real.T, Ytrain)
print("Accuracy with R = ", R[2], ": ", knn.score(projected_data7_real.T, Ytest))

knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(projected_data4_real.T, Ytrain)
print("Accuracy with R = ", R[3], ": ", knn.score(projected_data8_real.T, Ytest))
```

```
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py:215: D
  return self._fit(X, y)
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py:215: D
  return self._fit(X, y)
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py:215: D
  return self._fit(X, y)
Accuracy with R =  37.0 :  0.93
Accuracy with R =  53.0 :  0.94
Accuracy with R =  77.0 :  0.945
Accuracy with R =  116.0 :  0.935
/usr/local/lib/python3.9/dist-packages/sklearn/neighbors/_classification.py:215: D
  return self._fit(X, y)
```

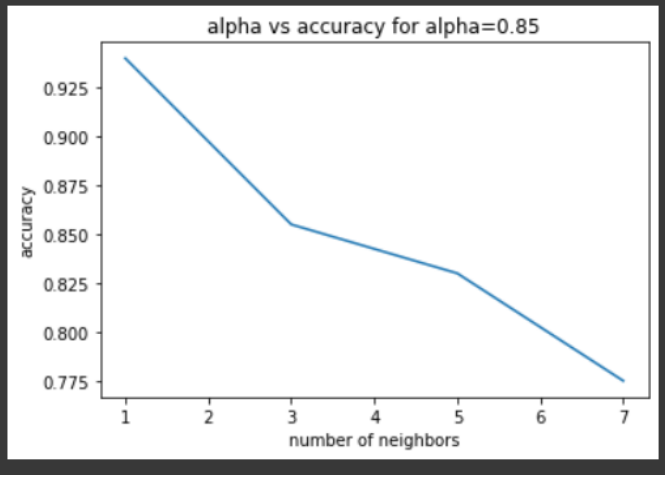
```
k_values = [1, 3, 5, 7]

accuracy_R0=np.zeros(4)
accuracy_R1=np.zeros(4)
accuracy_R2=np.zeros(4)
accuracy_R3=np.zeros(4)

for i in range(0,4):
  knn = KNeighborsClassifier(n_neighbors=k_values[i])
  k=np.zeros(4)
  knn.fit(projected_data1_real, Ytrain)
  accuracy_R0[i]=knn.score(projected_data5_real, Ytest)
  print("Accuracy with R = ", R[0], "and number of neighbors =",k_values[i],": ", accuracy_R0[i])

  knn = KNeighborsClassifier(n_neighbors=k_values[i])
  knn.fit(projected_data2_real, Ytrain)
  accuracy_R1[i]=knn.score(projected_data6_real, Ytest)
  print("Accuracy with R = ", R[1],"and number of neighbors =",k_values[i], ": ",  accuracy_R1[i] )

  knn = KNeighborsClassifier(n_neighbors=k_values[i])
  knn.fit(projected_data3_real, Ytrain)
  accuracy_R2[i]=knn.score(projected_data7_real, Ytest)
  print("Accuracy with R = ", R[2],"and number of neighbors =",k_values[i], ": ", accuracy_R2[i])

  knn = KNeighborsClassifier(n_neighbors=k_values[i])
  knn.fit(projected_data4_real, Ytrain)
  accuracy_R3[i]=knn.score(projected_data8_real, Ytest)
  print("Accuracy with R = ", R[3],"and number of neighbors =",k_values[i], ": ", accuracy_R3[i])
```

```
Accuracy with R =  37.0 and number of neighbors = 1 :  0.93
Accuracy with R =  53.0 and number of neighbors = 1 :  0.94
Accuracy with R =  77.0 and number of neighbors = 1 :  0.945
Accuracy with R =  116.0 and number of neighbors = 1 :  0.935
Accuracy with R =  37.0 and number of neighbors = 3 :  0.855
Accuracy with R =  53.0 and number of neighbors = 3 :  0.855
Accuracy with R =  77.0 and number of neighbors = 3 :  0.85
Accuracy with R =  116.0 and number of neighbors = 3 :  0.845
Accuracy with R =  37.0 and number of neighbors = 5 :  0.805
Accuracy with R =  53.0 and number of neighbors = 5 :  0.83
Accuracy with R =  77.0 and number of neighbors = 5 :  0.815
Accuracy with R =  116.0 and number of neighbors = 5 :  0.815
Accuracy with R =  37.0 and number of neighbors = 7 :  0.78
Accuracy with R =  53.0 and number of neighbors = 7 :  0.775
Accuracy with R =  77.0 and number of neighbors = 7 :  0.755
Accuracy with R =  116.0 and number of neighbors = 7 :  0.74
```

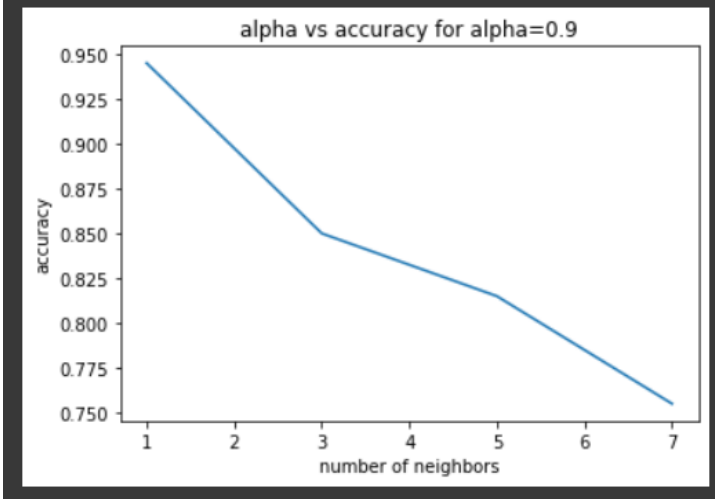
```python
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R0)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy for alpha=0.8')

# Show the plot
plt.show()
```



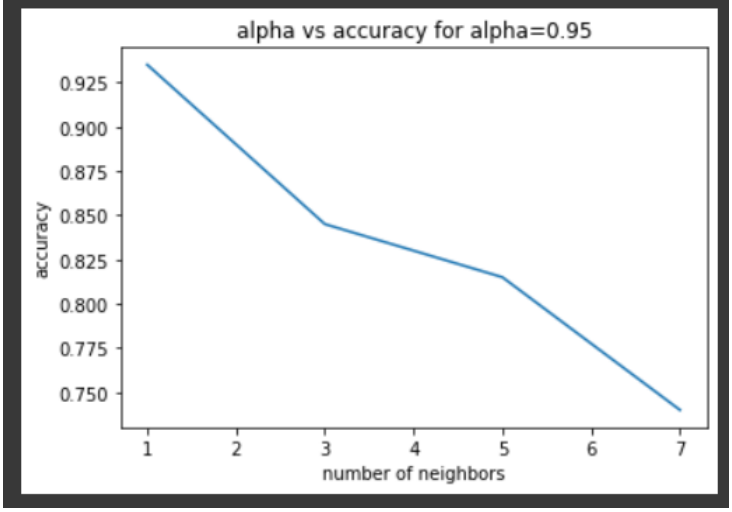

```python
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R1)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy for alpha=0.85')

# Show the plot
plt.show()
```



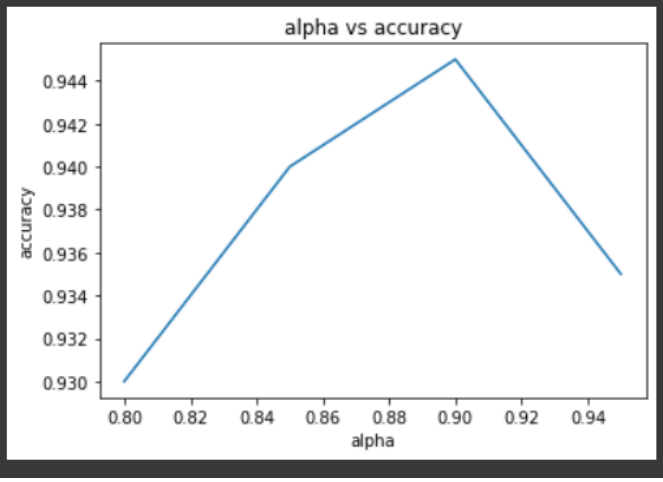
```python
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R2)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy for alpha=0.9')

# Show the plot
plt.show()
```




```python
import matplotlib.pyplot as plt
plt.plot(k_values,accuracy_R3)

# Set axis labels and title
plt.xlabel('number of neighbors')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy for alpha=0.95')

# Show the plot
plt.show()
```




```python
import matplotlib.pyplot as plt
plt.plot([0.8,0.85,0.9,0.95], [accuracy_R0[0],accuracy_R1[0],accuracy_R2[0],accuracy_R3[0]])

# Set axis labels and title
plt.xlabel('alpha')
plt.ylabel('accuracy')
plt.title('alpha vs accuracy')

# Show the plot
plt.show()
```



Part Two LDA:

```
#Bonus Part2 LDA
from PIL import Image
import numpy as np
import os
from sklearn.neighbors import KNeighborsClassifier
from sklearn.decomposition import IncrementalPCA

parent_folder = '/content/drive/MyDrive/PR/assignment1'

width = 92
height = 112
D = np.zeros((400, 10304))
DTraincentered = np.zeros((200, 10304))
DTestcentered = np.zeros((200, 10304))
testc=0
trainc=0
Dtrain = np.zeros((200, 10304))
Dtest = np.zeros((200, 10304))
Y = np.zeros(400)
Ytrain = np.zeros((200, 1))
Ytest = np.zeros((200, 1))
```

```
for sub in range(1,41):
  folder_path = f"{parent_folder}/s{sub}"
  for i in range(1,11):
    filename = f"{folder_path}/{i}.pgm"
    image = Image.open(filename)
    image_array = np.array(image)
    D[(sub-1)*10+(i-1)] = image_array.flatten()
    Y[(sub-1)*10+(i-1)] = sub
    if (i % 2) != 0:
      Dtest[testc] = D[(sub-1)*10+(i-1)]
      Ytest[int(testc)] = Y[(sub-1)*10+(i-1)]
      testc=testc+1
    else:
      Dtrain[trainc] = D[(sub-1)*10+(i-1)]
      Ytrain[int(trainc)] = Y[(sub-1)*10+(i-1)]
      trainc=trainc+1

for i in range(0,10304):
  DTraincentered[:,i]=Dtrain[:,i]-Dtrain[:,i].mean()

for i in range(0,10304):
  DTestcentered[:,i]=Dtest[:,i]-Dtrain[:,i].mean()
```

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
# Create the model with regularization
lda = LinearDiscriminantAnalysis(solver='eigen', shrinkage='auto')

# Fit the model to the training data
lda.fit(Dtrain, Ytrain)

# Predict the class labels for the testing data
y_pred = lda.predict(Dtest)

# Evaluate the performance of the model
accuracy = accuracy_score(Ytest, y_pred)
print("Accuracy: {:.2f}%".format(accuracy * 100))
```

```
/usr/local/lib/python3.9/dist-packages/sklearn/utils/validation.py:11
  y = column_or_1d(y, warn=True)
Accuracy: 98.50%
```

Comparisons:

- **Part 4 – e)** Can you find a relation between alpha and classification accuracy? (PCA Classification)

Answer:

- In the Simple Classifier: accuracy changes with the change of alpha.
  Where in alpha = 0.8, R = 37.0 so the accuracy = 93%
  In case of alpha = 0.85 -> R = 53.0 -> accuracy = 94%
  Alpha = 0.9 -> R = 77.0 -> accuracy = 94.5%
  Alpha = 0.95 -> R = 116.0 -> accuracy = 93.5%
- In the Classifier Tuning: accuracy is inversly proportional to the number of neighbours, where the accuracy decreases by increasing the number of neighbours with the same 4 alpha values.
  N = 1 , alpha = 0.8 -> R = 37 -> accuracy = 93%
  N = 1 , alpha = 0.85 -> R = 53 -> accuracy = 94%
  N = 1 , alpha = 0.9 -> R = 77 -> accuracy = 94.5%
  N = 1 , alpha = 0.95 -> R = 116 -> accuracy = 93.5%
  N = 3 , alpha = 0.8 -> R = 37 -> accuracy = 85.5%
  N = 3 , alpha = 0.85 -> R = 53 -> accuracy = 85.5%
  N = 3 , alpha = 0.9 -> R = 77 -> accuracy = 85%
  N = 3 , alpha = 0.95 -> R = 116 -> accuracy = 84.5%
  N = 5 , alpha = 0.8 -> R = 37 -> accuracy = 80.5%
  N = 5 , alpha = 0.85 -> R = 53 -> accuracy = 83%
  N = 5 , alpha = 0.9 -> R = 77 -> accuracy = 81.5%
  N = 5 , alpha = 0.95 -> R = 116 -> accuracy = 81.5%
  N = 7 , alpha = 0.8 -> R = 37 -> accuracy = 78%
  N = 7 , alpha = 0.85 -> R = 53 -> accuracy = 77.5%
  N = 7 , alpha = 0.9 -> R = 77 -> accuracy = 75.5%
  N = 7 , alpha = 0.95 -> R = 116 -> accuracy = 74%

- **Part 5 – e)** Compare the LDA results to the PCA results.

Answer:

P.O.C	**PCA**	**LDA**
Simple Classifier	α=0.8 -> accuracy=93% α=0.85 -> accuracy=94% α=0.9 -> accuracy=94.5% α=0.95 -> accuracy=93.5%	Accuracy = 93.5%
Classifier Tuning	With respect to alpha N=1 -> accuracy= 93% - 94.5% N=3 ->accuracy= 84.5% - 85.5% N=5 -> accuracy= 80.5% - 83% N=7 -> accuracy= 74% - 78%	N=1 -> accuracy=93.5% N=3 -> accuracy=86% N=5 -> accuracy=79.5% N=7 -> accuracy=77.5%

Therefore, the accuracy doesn't only depend on the type of analysis. It also depends on the dataset itself, the number of neighbours, and the alpha in the pca. So in some cases, the accuracy of PCA is better and in other cases the accuracy of LDA is better.

- **Part 8 – a)** Compare the results of 70% Training 30% Test split with the 50% split.

Answer: this comparison is done in the PCA Classification and the LDA Classification also.

The following table shows the comparison in all the cases:

P.O.C	50% split	30%-70% split
Simple Classifier PCA	α=0.8 -> accuracy=93% α=0.85 -> accuracy=94% α=0.9 -> accuracy=94.5% α=0.95 -> accuracy=93.5%	α=0.8 -> accuracy=95.8% α=0.85 -> accuracy=95.8% α=0.9 -> accuracy=95.8% α=0.95 -> accuracy=95.8%
Simple Classifier LDA	Accuracy = 93.5%	Accuracy = 95%
Classifier Tuning PCA	With respect to alpha N=1 -> accuracy= 93% - 94.5% N=3 ->accuracy= 84.5% - 85.5% N=5 -> accuracy= 80.5% - 83% N=7 -> accuracy= 74% - 78%	With respect to alpha N=1 -> accuracy= 95.8% N=3 ->accuracy= 90.8% - 92.5% N=5 -> accuracy= 85.8% - 88.3% N=7 -> accuracy= 80% - 87.5%

In conclusion, the 30% 70% split is better than the 50% split. Where its accuracy is higher than the 50% split.

- **Part 8 – b)** Compare PCA variation with the original algorithm.

Answer:

The PCA variation used is Incremental PCA, The difference between normal PCA and Incremental PCA is the way they handle large datasets.

Where incremental PCA is designed to handle large datasets that cannot fit into memory. IPCA processes the data in small batches and updates the covariance matrix incrementally. This approach reduces memory usage and computational complexity, making it more efficient for handling large datasets.

While in normal PCA the entire dataset is loaded into memory and the covariance matrix is computed for all the data points at once. This approach can be computationally expensive and memory-intensive, making it impractical for very large datasets.

P.O.C	**Normal PCA**	**Incremental PCA**
Simple Classifier	α=0.8 -> accuracy=93% α=0.85 -> accuracy=94% α=0.9 -> accuracy=94.5% α=0.95 -> accuracy=93.5%	α=0.8 -> accuracy=93% α=0.85 -> accuracy=94% α=0.9 -> accuracy=94.5% α=0.95 -> accuracy=93.5%
Classifier Tuning	With respect to alpha N=1 -> accuracy= 93% - 94.5% N=3 ->accuracy= 84.5% - 85.5% N=5 -> accuracy= 80.5% - 83% N=7 -> accuracy= 74% - 78%	With respect to alpha N=1 -> accuracy= 93% - 94.5% N=3 ->accuracy= 84.5% - 85.5% N=5 -> accuracy= 80.5% - 83% N=7 -> accuracy= 74% - 78%
Time Complexity	$O(n^3)$	$O(m*n^2)$

In conclusion, the results are close to each other because both PCA and IPCA use eigenvalue decomposition to identify these important features and compute a set of principal components that capture most of the variability in the data.

In both cases, the accuracy of PCA and IPCA will depend on several factors such as the number of principal components retained, the amount of variance explained, and the choice of parameters. If these factors are optimized appropriately, it is possible for the accuracies in PCA and IPCA to be the same.

- **Part 8 – b)** Compare LDA variation with the original algorithm.

Answer: The LDA variation used is Regularized LDA, the main difference between normal LDA and regularized LDA is that regularized LDA uses a regularization parameter to shrink the covariance matrix estimates, while normal LDA assumes that the covariance matrices are equal across classes and estimates them using the pooled within-class covariance matrix. Regularized LDA can be beneficial when the assumptions of LDA are not met or when the number of features is large compared to the number of samples.

LDA aims to identify a linear combination of features that maximizes the separation between classes. It assumes that the data is normally distributed and that the classes have equal covariance matrices. However, in some cases, the assumptions of LDA may not hold, leading to overfitting or poor performance.

Regularized LDA, also known as Shrinkage LDA, is a variant of LDA that uses a regularization parameter to overcome some of the limitations of LDA. The regularization parameter allows for some shrinkage of the covariance matrix estimates, which can improve the stability of the estimates and reduce overfitting.

P.O.C	**LDA**	**Regularized LDA**
Accuracy	Accuracy = 93.5% For N neighbours N=3 -> accuracy = 86% N=5 -> accuracy = 79.5% N=7 -> accuracy = 77.5%	Accuracy = 98.5%

Therefore, Regularized LDA is more effiecient than normal LDA.